Explain At Most Logic

Yes — this is a **very important sliding window pattern**. The whole trick is:

> **Exactly K distinct = At most K distinct − At most (K − 1) distinct**

Let's understand **why `atMost()` works**, because that is the real logic behind the solution.

---

1. What does `atMost(nums, k)` mean?

Your function:

```
C++
int atMost(vector<int>& nums, int k)
```

returns:

> **The total number of subarrays having at most `k` distinct elements.**

For example:

```
nums = [1, 2, 1]
```

All subarrays:

```
[1]        → 1 distinct
[1,2]      → 2 distinct
[1,2,1]    → 2 distinct
[2]        → 1 distinct
[2,1]      → 2 distinct
[1]        → 1 distinct
```

So:

```
atMost(nums, 1) = 3
atMost(nums, 2) = 6
```

2. Why do we need `atMost` for an "exactly K" problem?

Suppose we want:

```
Exactly K distinct elements
```

Think of all subarrays like this:

```
At most K
=
Exactly 1
+ Exactly 2
+ Exactly 3
+ ...
+ Exactly K
```

And:

```
At most (K - 1)
=
Exactly 1
+ Exactly 2
+ ...
+ Exactly (K - 1)
```

Now subtract:

```
At most K
-
At most (K - 1)
```

Everything from **`Exactly 1`** to **`Exactly K-1`** cancels.

Only this remains:

```
Exactly K
```

Mathematically

```
atMost(K)
=
```

```
exactly(1)
+ exactly(2)
+ ...
+ exactly(K)

atMost(K-1)
=
exactly(1)
+ exactly(2)
+ ...
+ exactly(K-1)
```

Therefore:

```
Exactly(K)
=
atMost(K) - atMost(K-1)
```

That is why your final code is:

```
C++
return atMost(nums, k) - atMost(nums, k - 1);
```

3. Now let's understand the `atMost()` logic

Your code:

```
C++
int atMost(vector<int>& nums, int k) {

    int right = 0;
    int left = 0;
    int count = 0;

    unordered_map<int, int> mpp;

    while (right < nums.size()) {

        mpp[nums[right]]++;

        while (mpp.size() > k) {

            mpp[nums[left]]--;

            if (mpp[nums[left]] == 0) {
                mpp.erase(nums[left]);
            }

            left++;
        }

        count += right - left + 1;

        right++;
    }

    return count;
}
```

The sliding window is:

```
[left ..... right]
```

And:

```
C++
mpp.size()
```

represents:

> Number of distinct elements currently inside the window.

---

4. First, expand the window

This line:

```
C++
mpp[nums[right]]++;
```

adds **`nums[right]`** into the current window.

For example:

```
nums = [1, 2, 1, 2, 3]
```

Initially:

```
left = 0
right = 0
```

Window:

```
[1]
```

Map:

```
{1 : 1}
```

Distinct elements:

```
mpp.size() = 1
```

5. What if distinct elements become more than K?

Suppose:

```
K = 2
```

Current window becomes:

```
[1, 2, 1, 2, 3]
```

Map:

```
{
    1 : 2,
    2 : 2,
    3 : 1
}
```

So:

```
mpp.size() = 3
```

But we only allow:

```
at most 2 distinct
```

Therefore:

```
C++
while (mpp.size() > k)
```

we shrink the window from the left.

---

6. Why decrease frequency first?

This:

```
C++
mpp[nums[left]]--;
```

means:

> Remove the left element from the window.

Suppose:

```
Window = [1, 2, 1, 2, 3]
          ↑
         left
```

Remove **1**:

```
Window = [2, 1, 2, 3]
```

Map changes:

```
Before:
```

```
1 -> 2
2 -> 2
3 -> 1
```

After removing one **1**:

```
1 -> 1
2 -> 2
3 -> 1
```

Still:

```
mpp.size() = 3
```

So we continue shrinking.

Eventually:

```
[3]
```

or:

```
[2, 3]
```

depending on the frequencies and elements.

The important condition is:

```
C++
mpp.size() <= k
```

Then the window becomes valid again.

---

7. The MOST IMPORTANT LINE

```
C++
count += right - left + 1;
```

This is where most people get confused.

Let's understand it carefully.

Suppose after shrinking, we have:

```
left = 2
right = 5
```

So the valid window is:

```
index:  2   3   4   5
        [ a   b   c   d ]
```

Because this whole window has:

```
<= K distinct elements
```

Then how many valid subarrays **end at `right`**?

They are:

```
[a, b, c, d]   → starts at 2
[b, c, d]      → starts at 3
[c, d]         → starts at 4
[d]            → starts at 5
```

Number of subarrays:

```
right - left + 1

= 5 - 2 + 1
= 4
```

Therefore:

```
C++
count += right - left + 1;
```

8. Why are ALL these subarrays valid?

This is the key observation.

If:

```
[left ......... right]
```

has at most `K` distinct elements, then removing elements from the left **cannot increase** the number of distinct elements.

For example:

```
Window:

[1, 2, 1, 3]
```

Suppose it has at most `3` distinct elements.

Then all suffixes are also valid:

```
[1, 2, 1, 3]
   [2, 1, 3]
      [1, 3]
         [3]
```

Removing elements can only:

```
decrease distinct elements
```

or:

```
keep them the same
```

It can never create a new distinct element.

Therefore, once the window is valid:

```
[left ... right]
```

all subarrays ending at `right` and starting anywhere from:

```
left to right
```

are automatically valid.

That is why:

```
C++
right - left + 1
```

works.

---

9. Complete dry run

Take:

```
nums = [1, 2, 1, 2, 3]
K = 2
```

We calculate:

```
atMost(nums, 2)
```

Step 1: right = 0

Window:

```
[1]
 ↑
 L,R
```

Map:

```
{1:1}
```

Distinct:

```
1 <= 2
```

Valid subarrays ending at index `0`:

```
[1]
```

Count:

```
count += 0 - 0 + 1
count = 1
```

Step 2: right = 1

Window:

```
[1, 2]
 L     R
```

Map:

```
{
  1:1,
  2:1
}
```

Distinct:

```
2 <= 2
```

Subarrays ending at `right = 1`:

```
[1,2]
[2]
```

Number:

```
1 - 0 + 1 = 2
```

Total:

```
count = 1 + 2 = 3
```

Step 3: right = 2

Window:

```
[1, 2, 1]
 L        R
```

Map:

```
{
  1:2,
  2:1
}
```

Distinct:

```
2 <= 2
```

Valid subarrays ending at index `2`:

```
[1,2,1]
[2,1]
[1]
```

Number:

```
2 - 0 + 1 = 3
```

Total:

```
count = 3 + 3 = 6
```

Step 4: right = 3

Window:

```
[1, 2, 1, 2]
 L        R
```

Map:

```
{
  1:2,
  2:2
}
```

Still:

```
2 distinct <= 2
```

New valid subarrays ending at index **3**:

```
[1,2,1,2]
[2,1,2]
[1,2]
[2]
```

Number:

```
3 - 0 + 1 = 4
```

Total:

```
count = 6 + 4 = 10
```

Step 5: right = 4

Add **3**:

```
[1, 2, 1, 2, 3]
 L           R
```

Map:

```
{
  1:2,
  2:2,
  3:1
}
```

Distinct:

```
3 > 2 ❌
```

So shrink.

Remove nums[left] = 1

```
[2, 1, 2, 3]
 L        R
```

Map:

```
{
  1:1,
  2:2,
  3:1
}
```

Still:

```
3 > 2
```

Shrink again.

Remove nums[left] = 2

```
[1, 2, 3]
 L        R
```

Map:

```
{
  1:1,
  2:1,
  3:1
}
```

Still:

```
3 > 2
```

Shrink again.

Remove nums[left] = 1

```
[2, 3]
 L     R
```

Map:

```
{
  2:1,
  3:1
}
```

Now:

```
2 <= 2 ✅
```

Current:

```
left = 3
right = 4
```

Valid subarrays ending at index **4**:

```
[2,3]
[3]
```

Number:

```
4 - 3 + 1 = 2
```

Final:

```
atMost(nums, 2)
=
1 + 2 + 3 + 4 + 2
=
12
```

10. Now calculate `atMost(nums, 1)`

This counts all subarrays with **at most 1 distinct element**.

For:

```
[1, 2, 1, 2, 3]
```

Only individual elements work:

```
[1]
[2]
[1]
[2]
[3]
```

So:

```
atMost(nums, 1) = 5
```

Therefore:

```
C++
subarraysWithKDistinct(nums, 2)
```

```
=
atMost(nums, 2)
-
atMost(nums, 1)
```

```
= 12 - 5
= 7
```

So there are:

```
7 subarrays with exactly 2 distinct elements
```

The complete intuition in one diagram

```
                    ALL SUBARRAYS
                         |
                         v

              ┌──────────────────────┐
              │   AT MOST K          │
              │                      │
              │ Exactly 1            │
              │ Exactly 2            │
              │ Exactly 3            │
              │ ...                  │
              │ Exactly K            │
              └──────────────────────┘
                         -
              ┌──────────────────────┐
              │   AT MOST K - 1      │
              │                      │
              │ Exactly 1            │
              │ Exactly 2            │
              │ ...                  │
              │ Exactly K - 1        │
              └──────────────────────┘
                         =
                         v

                    EXACTLY K
```

So the entire algorithm is based on:

```
C++
exactlyK = atMostK - atMostKMinus1;
```

And inside **`atMost()`**:

```
1. Expand right
        ↓
2. Add nums[right] to map
        ↓
3. If distinct > K
        ↓
   Move left until distinct <= K
        ↓
4. Count all valid subarrays ending at right
        ↓
   right - left + 1
```

The core idea to remember

```
AT MOST problems are easy with sliding window.

EXACTLY K problems are often converted into:

EXACTLY K
=
AT MOST K
-
AT MOST (K-1)
```

This pattern is extremely useful for many problems involving **exactly K distinct elements, exactly K odd numbers, exactly K special elements, etc.**